ZION EDGE GATEWAY

Benchmark Report v0.1.1

Security Audit + Performance Optimization Results

April 14, 2026

Apple M4 | Darwin arm64 | 10 cores | 16 GB RAM

233,341 req/s	Peak HTML throughput (TLS 1.3 e2e)
209,381 req/s	Cache hit throughput (4KB JS from RAM)
91,893 req/s	WAF POST throughput (70+ patterns)
28 bugs fixed	Critical, High, Medium severity
20 optimizations	Applied across all hot paths
0 errors	Across all benchmark runs

1. Executive Summary

Zion is a high-performance TLS reverse proxy with built-in Web Application Firewall (WAF), written entirely in Rust. This report covers the results of a comprehensive security audit (28 bugs fixed across 3 severity levels) and a performance optimization sprint (20 optimizations applied) on the v0.1.1 codebase.

The codebase was audited across all 17 modules (~8,600 lines of Rust) with focus on request smuggling, cache poisoning, WAF bypass vectors, memory safety, concurrency correctness, and RFC compliance. All fixes maintain backward compatibility with 154 unit tests passing.

2. Benchmark Results

Final benchmark run on commit **9c107f6** with all 28 security fixes and 20 performance optimizations applied. Native release build with target-cpu=native.

Endpoint	Median req/s	CV%	Best Run	Errors
HTML SSR 5KB	233,341	2.0%	236,755	0
Cache Hit JS 4KB (RAM)	209,381	9.8%	214,546	0
CSS 3KB (cached)	191,574	4.5%	203,969	0
TLS Proxy API GET 1KB	93,253	3.0%	97,019	0
WAF POST JSON	91,893	3.1%	93,415	0
JS 4KB (no cache)	81,470	2.3%	82,723	0
PNG 8KB (no cache)	66,753	2.7%	68,020	0
WOFF2 16KB (no cache)	59,262	3.0%	60,679	0

Security Check	Result
SQLi Injection (query parameter)	Blocked (HTTP 400)
XSS Injection (query parameter)	Blocked (HTTP 400)

Methodology: wrk 4.2.0, 2 threads, 100 connections, 5 runs x 10s each. Self-signed TLS 1.3 certs (rustls 0.23). Go test backend on localhost:9090. Release build: opt-level=3, LTO=fat, codegen-units=1, panic=abort, mimalloc allocator.

3. Security Audit

28 bugs identified and fixed across all severity levels. Every fix includes a regression test. All 154 tests pass after each commit.

Critical (7)

– Request smuggling via forwarded Transfer-Encoding header (proxy.rs)
– Cache poisoning: cache key ignored query string (dispatch.rs)
– WebSocket 101 response missing Sec-WebSocket-Accept (proxy.rs)
– WAF bypass via quadruple URL encoding, normalization only 3 passes (waf.rs)
– WAF POST/PUT/PATCH early return skipped CORS headers and metrics (dispatch.rs)
– Vary header check disabled cache for all gzip-enabled upstreams (dispatch.rs)
– HTTP/80 handler had no rate limiting or URI length check (main.rs)

High (8)

– L1/L2 cache coherence gap: stale data served for up to TTL duration (1 year default)
– DELETE request body bypassed WAF inspection entirely
– CORS preflight from disallowed origins proxied to upstream
– Cache hit missing Content-Encoding header (garbled gzip responses)
– SSRF detection only matched http:// prefix (missed https, hex/decimal IP encoding)
– EWMA latency update not atomic: lost updates under concurrent health checks
– TLS CERT_GENERATION counter used Relaxed ordering (stale certs on ARM/Graviton)
– Client cert fingerprint XOR fold mislabeled as SHA256 in comments

Medium (13)

– URI length check ignored query string; command injection patterns required spaces
– Path traversal only caught 3-level; Content-Type prefix matching too permissive
– Bearer token extraction case-sensitive (violated RFC 6750)
– JWKS refresh task died permanently on first HTTP client build failure
– auth_profile references not validated in config (panic at runtime)
– Connection timeout killed entire HTTP/2 mux and WebSocket connections
– TLS prewarm + watcher race condition on cert store
– setsockopt failures silently ignored; SNI cert dirs not watched
– CORS origin comparison case-sensitive (violated RFC 6454)

4. Performance Optimizations

20 optimizations implemented across 6 categories. Estimated cumulative impact: -700ns per request on hot path, 14 fewer atomic operations per histogram observation, O(1) cache LRU (was O(N)), and thundering herd protection.

Compiler / Build

– target-cpu=native via .cargo/config.toml - unlocks NEON/AES-CE on Apple Silicon
– PGO build script for additional 10-20% (benchmarks/bench-pgo.sh)

Allocation Elimination

– Traceparent: 3x format!() replaced with [u8;55] stack buffer (-500ns/req)
– CORS origin: HeaderValue clone instead of String allocation
– WAF content-type: borrow from parts.headers instead of pre-clone
– Cache key: Arc::from() direct instead of String intermediate

Lock / Contention Reduction

– WebSocket TLS config: OnceLock, built once instead of per-upgrade
– Metrics render: ArcSwap replaces RwLock (lock-free /metrics)
– Histogram observe: 3 atomics instead of 17 (non-cumulative differential buckets)
– HTTP builder: Arc wrap (ref-count bump instead of deep struct clone)

Data Structures

– L1 cache: O(1) LRU via index-based doubly-linked list (was O(N) VecDeque scan)
– Host validation: single-pass byte scan (was 8 separate contains() calls)
– CORS origin: FNV hash set O(1) lookup (was Vec linear scan)

WAF Pipeline

– SIMD pre-filter: memchr3 fast-reject before Aho-Corasick (-200-500ns clean bodies)
– Normalization iterations capped at 2 (was 7, convergence check handles early exit)
– Thread-local buffer shrink-to-fit above 64KB (prevents OOM under attack)

Innovative

– Request coalescing (singleflight): N concurrent cache misses = 1 upstream fetch
– Health probe inline fast-path: /healthz responds in ~1us, bypasses full pipeline
– SO_BUSY_POLL on Linux: spin-poll NIC queue for -5-15us p99 latency

5. Architecture Overview

Request Processing Pipeline

Client → TLS 1.3 Handshake → Rate Limit → Radix Route Lookup → WAF 6-Gate Pipeline → Dispatch (proxy / cache / stream / websocket) → Security Headers → Client

Key Design Choices

– Lock-free concurrency: ArcSwap for TLS hot-reload, DashMap for cache/rate-limit, sharded atomic counters for metrics

– Two-level cache: L1 thread-local (~5ns, O(1) LRU) + L2 shared DashMap (~30ns, TTL eviction) with generation-based coherence

– Zero-regex WAF: Aho-Corasick O(N) single-pass over 70+ patterns, SIMD pre-filter bypass for clean traffic, entropy analysis for obfuscated payloads

– Hardware-aware bootstrap: CPU affinity pinning, L1d cache sizing, AES-NI/NEON detection, SO_REUSEPORT, TCP_FASTOPEN, io_uring multishot accept

– Graceful shutdown: 30s connection drain via semaphore-based tracking

– Request coalescing (singleflight): prevents thundering herd on cache cold starts

Module Map

Module	Lines	Purpose
dispatch.rs	~700	Request pipeline, routing, cache, CORS, WAF gate
waf.rs	~1,150	6-gate WAF: Aho-Corasick, entropy, simd-json
config.rs	~1,050	TOML parsing, validation, radix router build
tls.rs	~640	rustls config, SNI resolution, hot-reload, pre-warm
proxy.rs	~560	Upstream forwarding, WebSocket, SSE streaming
cache.rs	~550	Two-level L1/L2 cache with O(1) LRU
metrics.rs	~580	Lock-free sharded counters, differential histogram
security.rs	~475	CORS (FNV O(1)), rate limiter, header hardening
main.rs	~770	Accept loop, TLS, graceful shutdown, health fast-path

6. Methodology

Benchmark tool	wrk 4.2.0 [kqueue], 2 threads, 100 concurrent connections
Runs	5 measurement runs x 10 seconds each, median reported
Statistics	Median, CI95 (1.96 x stderr), coefficient of variation (CV%)
TLS	Self-signed certificates, TLS 1.3, rustls 0.23, session tickets + 0-RTT
Backend	Go test server on localhost:9090 (1KB JSON, 5KB HTML, 4-16KB assets)
Build profile	opt-level=3, LTO=fat, codegen-units=1, panic=abort, strip=true
Allocator	mimalloc (global)
Platform	Apple M4, macOS 15, arm64, 10 cores, 16 GB RAM
Reproducibility	bash benchmarks/bench-native.sh

Commit History (this session)

9c107f6	perf: batch 5 - O(1) LRU, singleflight, Arc builder, SO_BUSY_POLL, PGO
ab0867a	perf: CORS O(1) FNV hash set + security header cleanup
1ae9d6c	perf: batch 4 - eliminate allocations, lock-free metrics, health fast-path
551dc05	perf: batch 1-3 - target-cpu, zero-alloc, SIMD pre-filter
be4c311	fix(bench): accept HTTP 400 as WAF block status
a7ea0fd	fix: patch 13 medium-severity bugs
b915f7b	fix(security): patch 8 high-severity bugs
95556a4	fix(security): patch 7 critical bugs